

Software Foundations



Understand what software is, how it works, and how it is built.

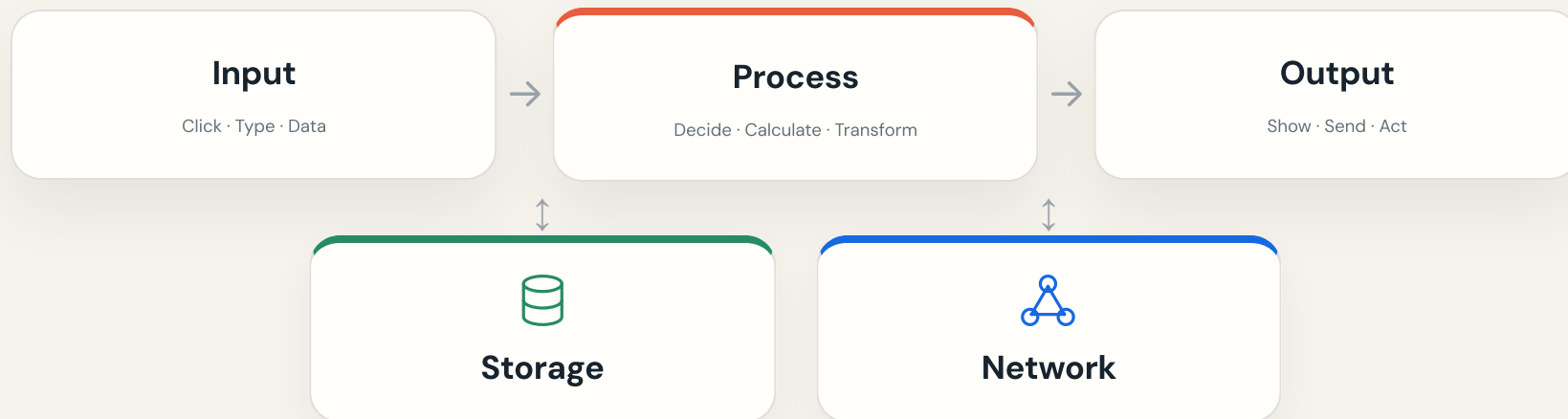
SECTION 1

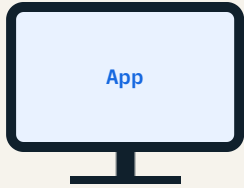
Types of software

Software can run on computers, mobile devices, and web browsers.

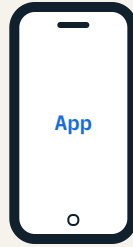
START HERE

Software is a system

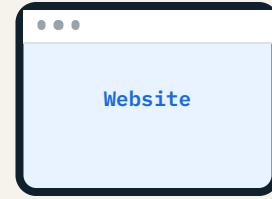




Desktop
Computer



Mobile
Phone or tablet



Web
Browser

DESKTOP SOFTWARE

Runs as an application on a computer

Usually installed for Windows, macOS, or Linux.

WRITE

Microsoft Word

CREATE

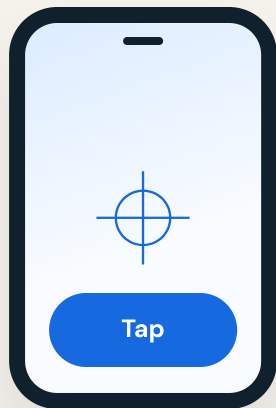
Photoshop

CODE

VS Code

Designed for phones and tablets

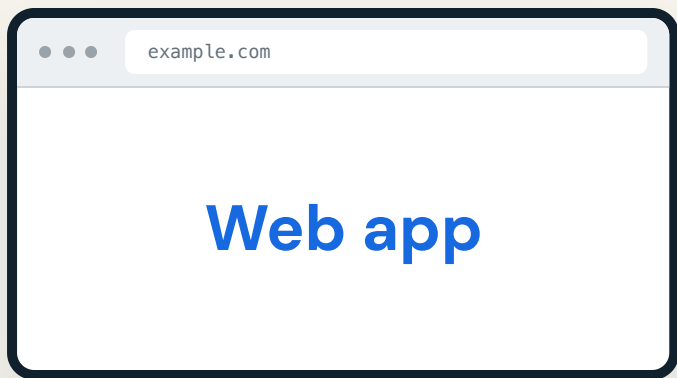
Touch, small screens, cameras, location, and notifications shape the experience.



- Touch
- Camera
- Location

Opens through a web browser

Visit an address. The browser loads the interface.

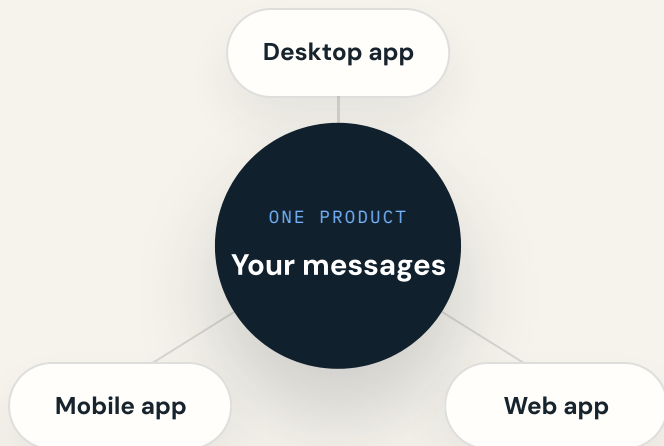


Update once

Reach many users

CATEGORIES CAN OVERLAP

One product. Three forms.





OTHER FORMS OF SOFTWARE

· **System software** · **Embedded software** · **Firmware**

Not all software appears as an app or website.

SECTION 2

Purpose of software

Ask who it is for and what it helps them do.

CONSUMER SOFTWARE

Built primarily for individuals

It helps a person communicate, learn, shop, create, or manage daily life.

COMMUNICATE

Messaging

ENJOY

Streaming

MANAGE

Personal finance

Helps organizations operate and decide

The customer is usually a company, school, nonprofit, or public organization.

SELL

Customer management

TRACK

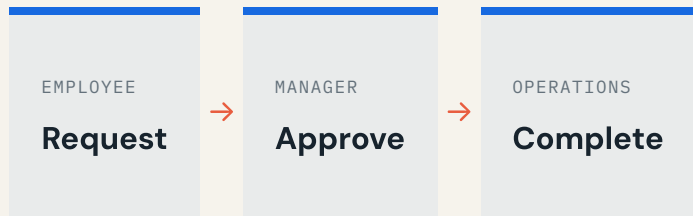
Inventory

OPERATE

Accounting

Built for one organization

Software can be built around one organization's workflow.



Some software helps build other software

Its users are developers, technical teams, or other programs.

Applications

↑ built with

Code editor

Database

Cloud service

Developer API

SECTION 3

Programming languages

Different languages make different tradeoffs.

CLOSE TO THE MACHINE

C · C++ · Rust

Common where control, performance, and system resources matter.

C

Operating systems · embedded software

C++

Games · engines · performance-heavy applications

Rust

Systems · command-line tools · infrastructure

Java · Kotlin · C#

Mature platforms support applications maintained by large teams.

Java

Enterprise backends · JVM applications

Kotlin

Android · JVM applications

C#

.NET applications · enterprise · games

THE WEB'S NATIVE LANGUAGE

JavaScript · TypeScript

From browser interfaces to servers, tools, and cross-platform applications.

JavaScript

Browser interfaces · Node.js servers

TypeScript

JavaScript with type information
Especially useful as a codebase grows

BROWSER → SERVER → TOOLS → APPS

PRODUCTIVE, EXPRESSIVE ECOSYSTEMS

Python · PHP · Ruby

Popular choices for automation, data work, and web backends.

Python

Automation · data · AI · web backends

PHP

Server-side web · content systems

Ruby

Web applications · scripting

Go · Swift · Dart

Languages often grow around a strong platform or development model.

Go

Cloud services · networking · command-line tools

Swift

Applications for Apple platforms

Dart

Flutter · multi-platform client applications

DIFFERENT JOBS IN THE STACK

HTML · CSS · SQL

Essential languages can solve narrower, specialized problems.

HTML

**Structure
and
meaning**

What is on the
page?

CSS

**Presentation
and layout**

How does it
look?

SQL

**Database
queries**

What data do
we need?

REAL PRODUCTS ARE SYSTEMS

One product can use several languages

Each part can choose technology that fits its job.

INTERFACE	JavaScript / TypeScript · Swift · Kotlin
SERVER	JavaScript / TypeScript · Java · C# · Python · Go
DATA	SQL
PERFORMANCE COMPONENT	C++ · Rust

SECTION 4

JavaScript / TypeScript

One ecosystem across the stack, with earlier feedback.

WHY JAVASCRIPT?

JavaScript works across web, mobile, server, and CLI

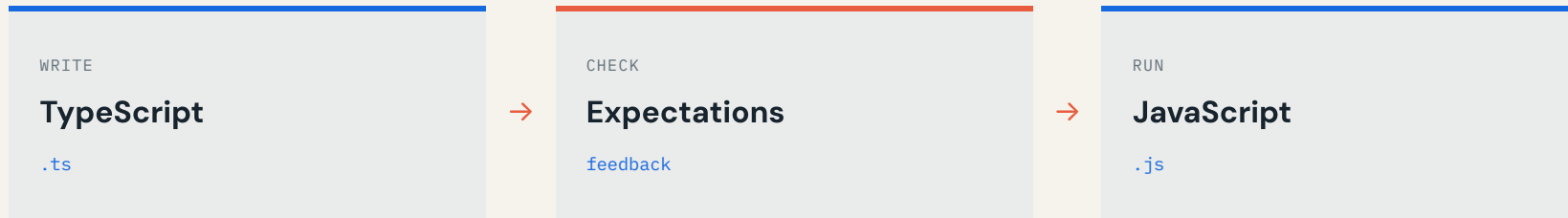
One language can support several kinds of software.

WEB + MOBILE + SERVER + CLI

WHY TYPESCRIPT?

TypeScript adds checkable expectations to JavaScript

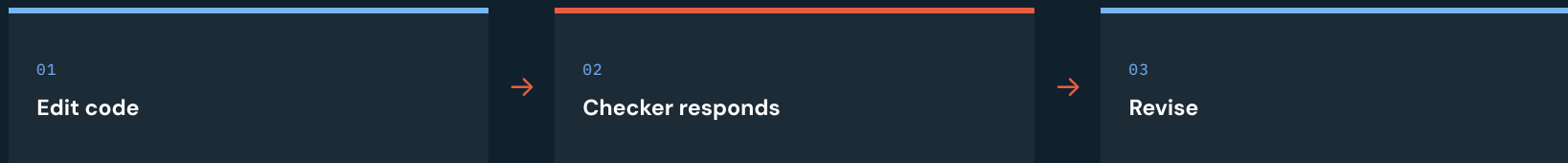
It checks code during development. JavaScript still runs the program.



FAST FEEDBACK

The checker guides the next edit

People and coding agents can use the same feedback loop.



IMPORTANT LIMIT

Types check consistency not correctness

We still need tests, validation, and review.

TYPES CAN CHECK

total is a number

≠

TYPES CANNOT PROVE

the total is correct

SECTION 5

Test code

Code that checks whether other code behaves as expected.

A TINY TEST

Give the code an input check the output

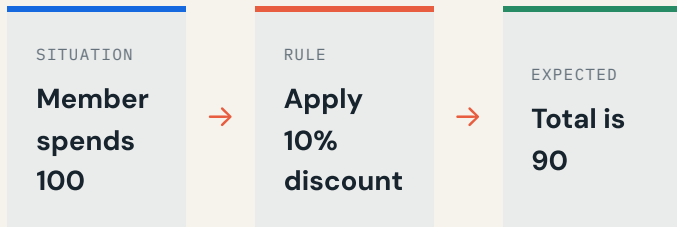
If $2 + 3$ should be 5, write that expectation down.

```
test('adds two numbers', () => {  
  expect(add(2, 3)).toBe(5)  
})
```

INPUT	2
	and
	3
EXPECTED	5

The example becomes executable documentation

It shows what the software should do in a specific situation.



TEST MORE THAN THE HAPPY PATH

Boundaries reveal hidden assumptions

Normal cases show that it works. Edge cases show where it may break.

NORMAL

2 items

Typical purchase

BOUNDARY

0 items

Empty cart

UNUSUAL

Large quantity

Limit or valid order?

A FAILED TEST IS USEFUL

It shows a mismatch between expected and actual

A failure is a signal to investigate—not a verdict about the developer.

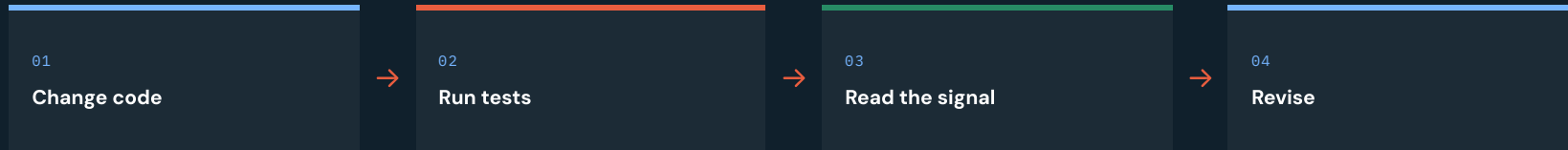


Which assumption changed?

WHY THIS HELPS AN AGENT

Tests turn expectations into observable feedback

An agent can run the suite, inspect failures, revise, and run it again.



Use both layers of feedback

Types check compatible data. Tests check selected behavior.

TYPE CHECK

Is total a number?

Structure and compatibility

+

TEST

Is the total 90?

Expected behavior

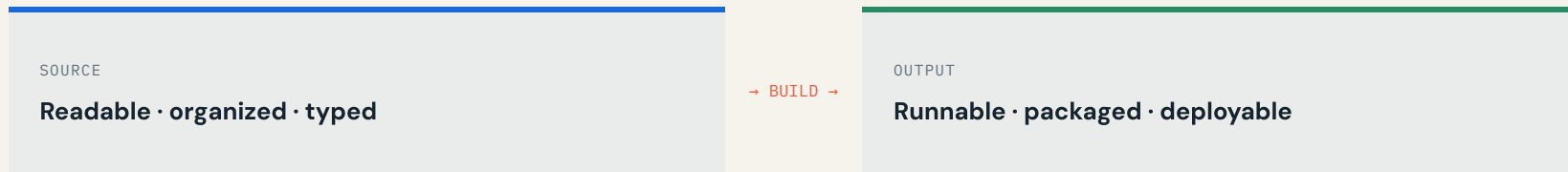
SECTION 6

Build

Turn source code into checked, runnable output.

SOURCE IS NOT ALWAYS THE FINAL PRODUCT

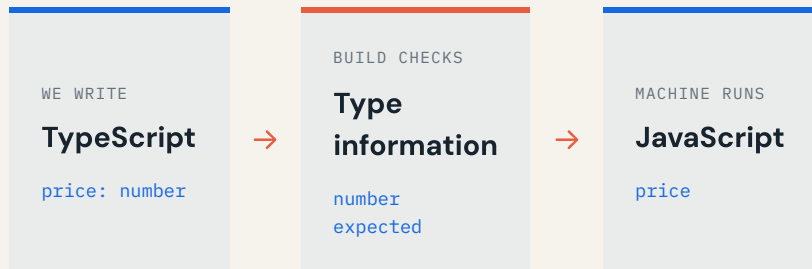
We write for people then prepare for machines



OUR CLEAREST EXAMPLE

TypeScript becomes JavaScript

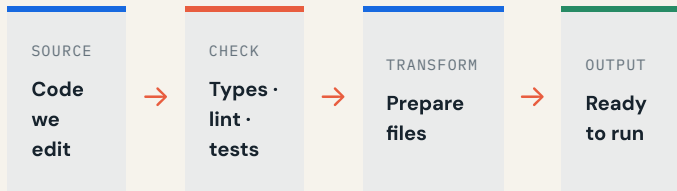
Types help during development; the JavaScript environment runs the result.



A BUILD IS A PIPELINE

Several steps can run behind one command

The project decides which checks and transformations belong in the pipeline.



WHAT IS A LINTER?

A linter inspects code for suspicious patterns

It flags likely mistakes and rules the team has agreed to follow.

```
CONST discount =  
10  
  
return total
```

LINT WARNING

**"discount" was
created but never
used.**

Maybe harmless. Maybe a
missing step.

DIFFERENT CHECKS FIND DIFFERENT ISSUES

Each tool asks a different question

Combine the signals instead of expecting one tool to catch everything.

TYPE CHECKER

**Do the data
shapes
agree?**

Text used where a
number is
expected

LINTER

**Does the
code look
suspicious?**

A value is created
but never used

TEST RUNNER

**Does
behavior
match
examples?**

Expected 90,
received 100

A BUILD CAN BECOME A QUALITY GATE

If a required check fails stop before handoff

Fix the signal before packaging or deploying the change.

CHANGE → BUILD → **CHECK FAILED** ↻ REVISE

SECTION 7

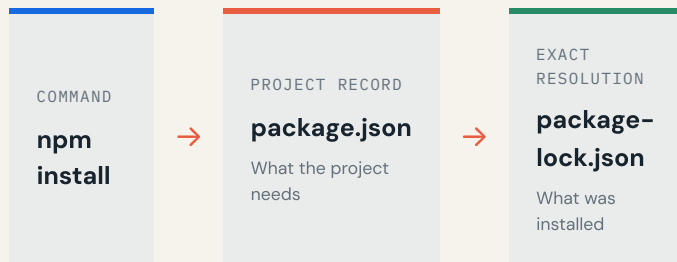
Packages

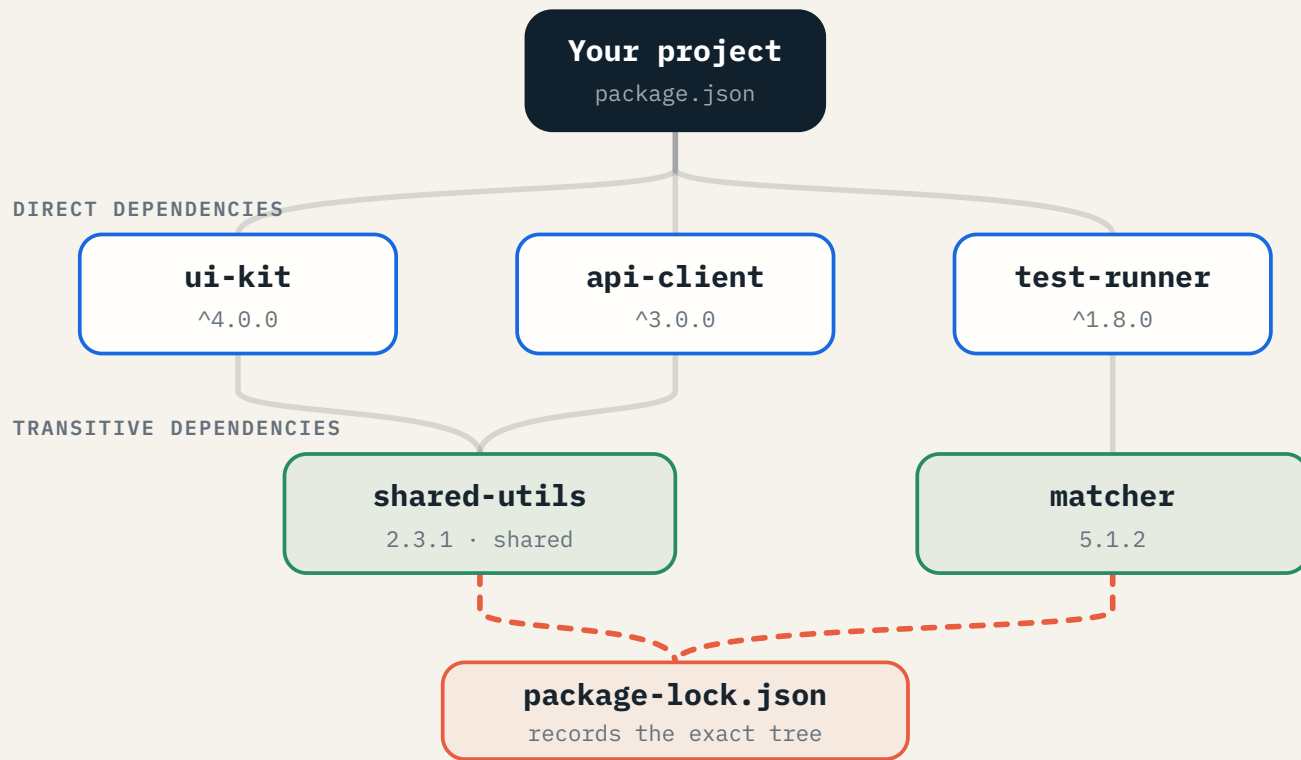
Reuse existing code instead of rebuilding everything.

THE PACKAGE MANAGER COORDINATES REUSE

Install · Record · Run

In this course, npm manages packages and project commands.





SECTION 8

Source control

Record intentional checkpoints as software changes.

